

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Industry Problem Solving Task

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 2 – Industry Problem Solving Task
Weightage:	20% of CIA	Total Marks:	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	Naveen Kumar S	Reg. No.:	192421030
Section / Batch:	B Tech IT	Date:	21.08.2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given questions.
- Explain in detail with sample code if needed.
- Clearly identify all key issues.
- Provide a thorough analysis with validated results and performance evaluation.

Question Type	Focus Area	Questions
Industry Problem Solving Task	Focus on Solving optimization problems using dynamic programming and greedy strategies	Q1

SECTION A – Industry Problem Solving Task

Code of Conduct

I Naveen Kumar S - 192421030 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Naveen Kumar S

RUBRICS FOR CALCULATION

Industry Problem Solving Task

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
----------	-----------	---------------------	----------------------	----------------------	---------------------

Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%				
Application of Engineering Concepts	25%				
Solution Design	25%				
Use of Tools	15%				
Analysis	10%				
Professionalism	5%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____



SIMATS Online Programming Lab

DAA PROGRAMMING


NAVEEN KUMAR S
192421030RC

```

2  n = len(demand)
3  # dp[i][j] = minimum cost to satisfy demand for days i..n-1 with
4  dp = [[Float('inf')] * (capacity + 1) for _ in range(n + 1)]
5
6  # Base case: no more days left, no cost
7  for j in range(capacity + 1):
8      dp[n][j] = 0
9
10 # Fill DP table from last day backwards
11 for i in range(n - 1, -1, -1):
12     for stock in range(capacity + 1):
13         # Try all possible order quantities
14         for order in range(capacity + 1 - stock):
15             available = stock + order
16             if available >= demand[i]:
17                 remaining = available - demand[i]
18                 cost = order_cost if order > 0 else 0
19                 cost += remaining * holding_cost
20                 total = cost + dp[i + 1][remaining]
21             else:
22                 shortage = demand[i] - available
23                 cost = order_cost if order > 0 else 0
24                 cost += shortage * shortage_cost
25                 total = cost + dp[i + 1][0]
26
27         if total < dp[i][stock]:
28             dp[i][stock] = total

```

Output

```

Demand per day: [10, 20, 15, 25, 10]
Holding cost per unit: 2
Shortage cost per unit: 5
Order cost per replenishment: 10
Storage capacity: 30

```